Containerization & DevOps Lab

Document: Lab1.md | Containerization & DevOps Coursework

Lab Report: Comparison of Virtual Machines (VMs) and Containers

1. Objective

The primary goals of this experiment are:
* To understand the conceptual and practical differences between Virtual Machines (VMs) and Containers.
* To install and configure an Ubuntu-based Nginx web server using both VirtualBox/Vagrant and Docker inside WSL.
* To compare resource utilization, performance, and operational characteristics of both environments.

2. Hardware and Software Requirements

Hardware

- 64-bit system with virtualization support enabled in BIOS.
- Minimum 8 GB RAM (4 GB acceptable).
- Internet connection.

Software

- Oracle VirtualBox and Vagrant.
- Windows Subsystem for Linux (WSL 2) with Ubuntu distribution.
- Docker Engine (docker.io).

3. Theory

Feature	Virtual Machine (VM)	Container
Virtualization Level	Emulates complete hardware and kernel.	Virtualizes at the OS level, sharing the host kernel.
Isolation	Strong (Full OS isolation).	Moderate (Process-level isolation).
Resource Usage	Higher (Requires dedicated RAM/CPU for guest OS).	Lightweight and efficient.
Startup Time	Slower (Minutes/Seconds).	Fast (Milliseconds/Seconds).

---

4. Experiment Setup - Part A: Virtual Machine

Step 1: Initialization and Deployment

Using Vagrant, an Ubuntu VM was initialized and started.
* **Command:** `vagrant init ubuntu/jammy64` followed by `vagrant up`.

```
PS C:|users|Yuvraj Singh> mkdir vm-Lab


    Directory: C:Iusers|Yuvraj Singh


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        31-01-2026     23:15                vm-lab


PS C:Iusers|Yuvraj Singh> cd vm-Lab
PS C: |Users |Yuvraj Singhlvm-Lab> vagrant init ubuntu/ jammy64
A `Vagrantfile` has been placed in this directory. You are now
ready to `vagrant up` your first virtual environment! Please read
the comments in the Vagrantfile as well as documentation on
`vagrantup.com` for more information on using Vagrant.
PS C:Iusers|Yuvraj Singh vm-Lab> vagrant up
Bringing machine 'default' up with 'virtualbox' provider...
==> default: Importing base box 'ubuntu/jammy64'...
==> default: Matching MAC address for NAT networking...
==> default: Checking if box 'ubuntu/jammy64' version '20241002.0.0' is up to date...
==> default: Setting the name of the VM: vm-lab_default_1769882140559_18457
==> default: Clearing any previously set network interfaces...
==> default: Preparing network interfaces based on configuration...
    default: Adapter 1: nat
==> default: Forwarding ports...
    default: 22 (guest) => 2222 (host) (adapter 1)
```

> **Observation:** The system downloads the base box (Ubuntu Jammy) and configures the VirtualBox provider. Port forwarding (2222 -> 22) is established.

Step 2: Accessing the VM (SSH)

Once the VM was up, we established a connection to the guest OS.
* **Command:** `vagrant ssh`

```
PS C: Users Yuvraj Singh vm-lab> vagrant ssh
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 5.15.0-164-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 System information as of Sat Jan 31 18:08:05 UTC 2026

  System load:             0.55
  Usage of /:              4.5% of 38.70GB
  Memory usage:            32%
  Swap usage:              0%
  Processes:               111
  Users logged in:         0
  IPv4 address for enp0s3: 10.0.2.15
  IPv6 address for enp0s3: fd00::28:41ff:fee4:55a4


Expanded Security Maintenance for Applications is not enabled.

1 update can be applied immediately.
1 of these updates is a standard security update.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status


The list of available updates is more than a week old.
```

)

> **Observation:** Successful login to the Ubuntu 22.04.5 LTS environment.

Step 3: Installing Nginx

Inside the VM terminal, the package lists were updated, and the Nginx web server was installed.
* **Commands:** `sudo apt update`, `sudo apt install -y nginx`

```
vagrant@ubuntu-jammy:~$ sudo apt update
sudo apt install -y nginx
sudo systemctl start nginx
Hit:1 http://security.ubuntu.com/ubuntu jammy-security InRelease
Hit:2 http://archive.ubuntu.com/ubuntu jammy InRelease
Hit:3 http://archive.ubuntu.com/ubuntu jammy-updates InRelease
Hit:4 http://archive.ubuntu.com/ubuntu jammy-backports InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
19 packages can be upgraded. Run 'apt list --upgradable' to see them.
Waiting for cache lock: Could not get lock /var/lib/dpkg/lock-frontend. It is held by process 2439 (unattended-upgr)...
Waiting for cache lock: Could not get lock /var/lib/dpkg/lock-frontend. It is held by process 2439 (unattended-upgr)

Waiting for cache lock: Could not get lock /var/lib/dpkg/lock-frontend. It is held by process 2439 (unattended-upgr)...
Waiting for cache lock: Could not get lock /var/lib/dpkg/lock-frontend. It is held by process 2439 (unattended-upgr)

Waiting for cache lock: Could not get lock /var/lib/dpkg/lock-frontend. It is held by process 2439 (unattended-upgr)...
Waiting for cache lock: Could not get lock /var/lib/dpkg/lock-frontend. It is held by process 2439 (unattended-upgr)

Waiting for cache lock: Could not get lock /var/lib/dpkg/lock-frontend. It is held by process 2439 (unattended-upgr)...
Waiting for cache lock: Could not get lock /var/lib/dpkg/lock-frontend. It is held by process 2439 (unattended-upgr)

Waiting for cache lock: Could not get lock /var/lib/dpkg/lock-frontend. It is held by process 2439 (unattended-upgr)...
Waiting for cache lock: Could not get lock /var/lib/dpkg/lock-frontend. It is held by process 2439 (unattended-upgr)

Waiting for cache lock: Could not get lock /var/lib/dpkg/lock-frontend. It is held by process 2439 (unattended-upgr)...
```

> **Observation:** The `apt` package manager retrieves necessary archives. This process is slower than Docker as it installs dependencies for a full OS environment.

Step 4: Verification Inside VM

We verified the server was running locally within the guest OS.
* **Command:** `curl localhost`

```
vagrant@ubuntu-jammy:~$ curl localhost
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
    body {
        width: 35em;
        margin: 0 auto;
        font-family: Tahoma, Verdana, Arial, sans-serif;
    }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
```

> **Observation:** The `curl` command inside the VM returns the full HTML source of the "Welcome to nginx!" page.

---

5. Experiment Setup - Part B: Containers (Docker)

Step 1: Running the Container

The Docker engine was used to pull the Ubuntu image and deploy a containerized Nginx instance.
* **Command:** `docker run -dp 8080:80 --name nginx-container nginx`

```
PS C: |Users|Yuvraj Singh vm-Lab> wsl
yuvraj_singh@LUCIF3R 'mnt/c/Users/Yuvraj Singh/vm-Labf docker pull ubuntu
Using default tag: latest
latest: Pulling from library/ubuntu
a3629ac5b9f4: Pull complete
Digest: sha256:cd1dba651b3080c3686ecf4e3c4220f026b521fb76978881737d24f200828b2b
Status: Downloaded newer image for ubuntu:latest
docker.io/librarv/ubuntu:latest
yuvraj_singh@LUCIF3R 'mnt/c/Users/Yuvraj Singh/vm-Labf docker run ~d ~p 8080: 80 ~-name nginx-container nginx
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
119d43eec815: Pull complete
700146c8ad64: Pull complete
d989100b8a84: Pull complete
500799c30424: Pull complete
10b68cfefee1: Pull complete
57f0dd1befe2: Pull complete
eaf8753feae0: Pull complete
Digest: sha256:c881927c4077710ac4b1da63b83aa163937fb47457950c267d92f7e4dedf4aec
Status: Downloaded newer image for nginx:latest
2b4aca5cf3b2631be5b89041c077ce31f7db9189fc06dea8f7e47b15be962c45
```

> **Observation:** Docker pulls the image layers and starts the container nearly instantaneously.

Step 2: Verification

The Nginx server was verified by accessing the mapped port on the localhost.

* **Command:** `curl localhost:8080`

```
anirudh_chawla@LUCIF3R:/mnt/c/Users/Anirudh Chawla/vm-lab$ curl localhost:8080
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
```

> **Observation:** The `curl` command confirms the Nginx "Welcome" page is active on port 8080.

---

6. Resource Utilization & Comparison

This section uses specific metrics captured during the experiment to contrast the two technologies.

A. Boot Time Analysis

- **Metric:** Time taken to reach a usable state.
- **VM Command:** `systemd-analyze`

```
vagrant@ubuntu-jammy:~$ systemd-analyze
Startup finished in 6.948s (kernel) + 29.871s (userspace) = 36.819s
graphical.target reached after 27.730s in userspace
```

> **Observation (VM):** The VM took **36.819 seconds** to finish startup (6.9s kernel + 29.8s userspace).
> **Observation (Container):** The container started in **less than 1 second** (refer to Docker output in Sec 5).

B. Process Overhead & Isolation

- **Metric:** Number of background processes required to run the application.
- **VM Command:** `htop`

```
  0[                                                      0.0%]   Tasks: 29, 25 thr; 1 running
  1[                                                      0.0%]   Load average: 0.01 0.18 0.16
Mem[||||||||||||||||||||||||||||||||||||||||||||||||||| 194M/957M]   Uptime: 00:09:43
Swp[                                                      0K/0K]

    PID USER      PRI  NI  VIRT   RES   SHR S CPU%▽MEM%   TIME+  Command
    689 syslog     20   0  217M  5344  4272 S  0.0  0.5  0:00.04 /usr/sbin/rsyslogd -n -iNONE
    690 root       20   0 82700  3876  3536 S  0.0  0.4  0:00.00 /usr/sbin/irqbalance --foreground
    693 root       20   0 1367M 37756 23944 S  0.0  3.9  0:02.13 /usr/lib/snapd/snapd
    694 root       20   0 23696  7200  6240 S  0.0  0.7  0:00.20 /lib/systemd/systemd-logind
    696 root       20   0 1367M 37756 23944 S  0.0  3.9  0:00.55 /usr/lib/snapd/snapd
    697 root       20   0 1367M 37756 23944 S  0.0  3.9  0:00.19 /usr/lib/snapd/snapd
    698 root       20   0 1367M 37756 23944 S  0.0  3.9  0:00.00 /usr/lib/snapd/snapd
    699 root       20   0 1367M 37756 23944 S  0.0  3.9  0:00.00 /usr/lib/snapd/snapd
    705 syslog     20   0  217M  5344  4272 S  0.0  0.5  0:00.01 /usr/sbin/rsyslogd -n -iNONE
    706 syslog     20   0  217M  5344  4272 S  0.0  0.5  0:00.00 /usr/sbin/rsyslogd -n -iNONE
    707 syslog     20   0  217M  5344  4272 S  0.0  0.5  0:00.00 /usr/sbin/rsyslogd -n -iNONE
    736 root       20   0  6220  1108  1020 S  0.0  0.1  0:00.02 /sbin/agetty -o -p -- \u --keep-baud 115200,57600,38400,9600 ttyS0 vt220
    750 root       20   0  6176  1104  1016 S  0.0  0.1  0:00.03 /sbin/agetty -o -p -- \u --noclear tty1 linux
    761 root       20   0  107M 21312 13192 S  0.0  2.2  0:00.20 /usr/bin/python3 /usr/share/unattended-upgrades/unattended-upgrade-shutdown --wait-for-sign
    769 root       20   0 1367M 37756 23944 S  0.0  3.9  0:00.08 /usr/lib/snapd/snapd
    770 root       20   0 1367M 37756 23944 S  0.0  3.9  0:00.20 /usr/lib/snapd/snapd
    771 root       20   0  107M 21312 13192 S  0.0  2.2  0:00.00 /usr/bin/python3 /usr/share/unattended-upgrades/unattended-upgrade-shutdown --wait-for-sign
    783 root       20   0 1367M 37756 23944 S  0.0  3.9  0:00.25 /usr/lib/snapd/snapd
    842 root       20   0 15440  8884  7248 S  0.0  0.9  0:00.04 sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups
    856 root       20   0 1367M 37756 23944 S  0.0  3.9  0:00.26 /usr/lib/snapd/snapd
    862 root       20   0 1367M 37756 23944 S  0.0  3.9  0:00.24 /usr/lib/snapd/snapd
   1881 root       20   0 16924  9868  7716 S  0.0  1.0  0:00.03 sshd: vagrant [priv]
   1884 vagrant    20   0 17076  9316  7724 S  0.0  1.0  0:00.10 /lib/systemd/systemd --user
   1885 vagrant    20   0  102M  4780     0 S  0.0  0.5  0:00.00 (sd-pam)
   1948 vagrant    20   0 17472  7824  5300 S  0.0  0.8  0:00.50 sshd: vagrant@pts/0
   1949 vagrant    20   0  9156  5016  3272 S  0.0  0.5  0:00.05 -bash
   2271 root       20   0  289M 19444 16524 S  0.0  2.0  0:00.07 /usr/libexec/packagekitd
   2272 root       20   0  289M 19444 16524 S  0.0  2.0  0:00.00 /usr/libexec/packagekitd
   2273 root       20   0  289M 19444 16524 S  0.0  2.0  0:00.00 /usr/libexec/packagekitd
   2275 root       20   0  229M  6804  6176 S  0.0  0.7  0:00.02 /usr/libexec/polkitd --no-debug
   2276 root       20   0  229M  6804  6176 S  0.0  0.7  0:00.00 /usr/libexec/polkitd --no-debug
   2278 root       20   0  229M  6804  6176 S  0.0  0.7  0:00.00 /usr/libexec/polkitd --no-debug
   2824 root       20   0 55200 12020 10428 S  0.0  1.2  0:00.02 nginx: master process /usr/sbin/nginx -g daemon on; master_process on;
   2827 www-data   20   0 55872  5400  3484 S  0.0  0.6  0:00.00 nginx: worker process
   2828 www-data   20   0 55872  5400  3484 S  0.0  0.6  0:00.00 nginx: worker process
   2931 vagrant    20   0  8708  4468  3412 R  0.0  0.5  0:00.10 htop
F1Help  F2Setup F3SearchF4FilterF5Tree  F6SortByF7Nice -F8Nice +F9Kill  F10Quit
```

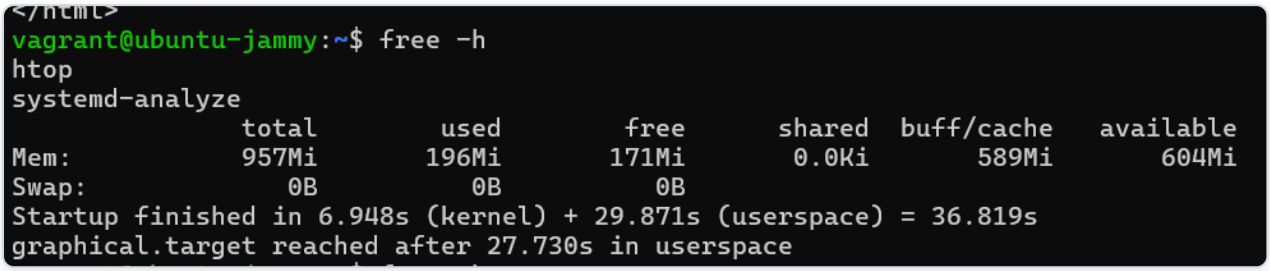
> **Observation (VM):** `htop` reveals a heavy process tree. Even though we only want Nginx, the VM is running `systemd`, `snapd`, `rsyslogd`, `polkitd`, and `sshd`. There are dozens of tasks running to support the OS.

```
CONTAINER ID   NAME              CPU %     MEM USAGE / LIMIT     MEM %     NET I/O         BLOCK I/O     PIDS
aca20b894db3   nginx-container   0.00%     13.22MiB / 7.592GiB   0.17%     2.02kB / 1.4kB   0B / 12.3kB   17
```

> **Observation (Container):** `docker stats` shows the container uses minimal resources because it *only* runs the application process (Nginx) and its direct dependencies.

C. Memory Usage

- **Metric:** RAM consumption.



> **Observation (VM):** The `free -h` command inside the VM shows it has allocated **957Mi** total, with **196Mi** used immediately by the OS kernel and services.
>
> **Observation (Container):** Referring to the Docker Stats image above, the container consumes only **13.22MiB** of RAM.

Comparison Summary Table

Parameter	Virtual Machine (VM)	Container (Docker)
Boot Time	**~36.8 seconds** (Measured)	**< 1 second**
RAM Usage	**~196 MiB** (Base OS overhead)	**~13.22 MiB** (App only)
Background Processes	High (init, systemd, ssh, logs)	Low (only Nginx entrypoint)
Disk Usage	Larger (Full OS libraries)	Smaller (Layered Image)

---

7. Conclusion

The experiment validates that **Containers** are significantly more lightweight.
1. **Speed:** The Docker container started instantly, whereas the VM required nearly 37 seconds to boot.
2. **Efficiency:** The VM required ~200MB of RAM just to exist, while the containerized application ran comfortably on ~13MB.
3. **Complexity:** The `htop` analysis clearly shows that VMs run a full operating system stack, creating unnecessary overhead for single-application deployments.

Therefore, Containers are ideal for microservices and rapid scaling, while VMs are better suited for scenarios requiring full hardware simulation or complete OS isolation.

---

8. Viva Voce Answers

1. **Main Difference:** VMs virtualize hardware (slow, heavy); containers virtualize the OS (fast, light).
2. **Fast Startup:** Containers use the host kernel and don't need to boot a new OS, saving the ~30s kernel/userspace init time seen in the VM.
3. **Hypervisor Role:** The hypervisor (VirtualBox) manages the VM's access to hardware.
4. **Different Kernels:** Containers share the host kernel. VMs run their own kernel.
5. **Why use `htop`?** To visualize the process tree and see that a VM runs many system services (systemd, journald) that a container does not.
